**FIAP - Tech Challenge – Fase 1 – Grupo 203**

Douglas Deveza dos Santos RM373827

Gabriel Pinelli Silva RM373763

João Carlos da Silva Brito RM371738

João Gabriel da Cruz Sales RM372444

João Vitor de Jesus Ciardullo RM372155

**Link do vídeo:** https://www.youtube.com/watch?v=MkTwbJPz4j0

**Documentação**

**Desafios encontrados e decisões tomadas**

O ToggleMaster foi desenvolvido como um monolito para priorizar simplicidade e rapidez no MVP, centralizando código, execução e banco de dados. Essa decisão facilitou o desenvolvimento, deploy e depuração, mas trouxe desafios como alto acoplamento, dificuldade de manutenção e limitação de escalabilidade, já que não é possível expandir partes isoladas.

Também foram identificadas falhas em segurança (credenciais expostas), ausência de pipeline CI/CD estruturado e limitações em observabilidade, com logs apenas em stdout. A aplicação adota parcialmente boas práticas do 12-Factor App methodology.

Foram tomadas decisões no sentido de melhorar a aplicação, com o uso de variáveis de ambiente para configurações de segurança, configuração de VPC e uso de binding do Gunicorn permitindo que a API escute a porta correta.

**Discussão sobre os 12-Factor App aplicada ao projeto.**

O ToggleMaster MVP apresenta uma arquitetura adequada para validação inicial da proposta, seguindo parcialmente os princípios do 12-Factor App. Entretanto, para evolução em ambientes de produção, são necessários ajustes relacionados à segurança, escalabilidade, observabilidade e automação de processos.

**1. Base de Código:** Uma base de código com rastreamento utilizando controle de versão e muitos deploys.

A aplicação atende a este fator, pois possui um único repositório versionado. Para produção pode ser necessário a utilização de estratégias de versionamento mais robustas, como branching e uso de tags.

**2. Dependências:** declarar e isolar dependências explicitamente.

A aplicação atende a este fator, pois as dependências estão declaradas em um único arquivo o requirements.txt. Para melhoria, poderia ser usado um lockfile e alguma ferramentas de verificação de vulnerabilidades.

**3. Configurações:** armazenar configuração em variáveis de ambiente.

A aplicação utiliza variáveis de ambiente, porém ainda há credenciais expostas. É necessário utilizar mecanismos seguros para gerenciamento de senhas como secrets ou variáveis injetadas pelo ambiente de implantação.

**4. Serviços de apoio:** tratar serviços externos como recursos vinculáveis.

O banco de dados Postgres é tratado como serviço externo no no docker-compose para ambiente de desenvolvimento. Em produção, seria melhor o uso de serviços gerenciados como o RDS com credenciais externas e configurações de conexões/timeout. Também seria melhor parametrizar URIs em vez de valores fixos.

**5. Construa, lance, execute:** separar fase de build, release e run.

Existe separação básica entre build (Dockefile) e execução (docker-compose), porém não há um processo formal de release. Portanto em produção seria melhor criar uma pipeline CI que produza imagens versionadas (build), gere release artifacts (imagem+env), e execute migrations no momento de release com rollback planejado.

**6. Processos:** executar como um ou mais processos stateless; persistência em serviços externos.

A aplicação segue o modelo stateless, mas ainda apresenta dependência de volumes locais em ambiente de desenvolvimento. Em produção seria bom fazer uso de RDBMS/volumes gerenciados para persistência.

**7. Vínculo de porta:** o app se expõe via porta.

A aplicação expõe sua funcionalidade via porta HTTP, atendendo ao princípio estabelecido.

**8. Concorrência:** processo múltiplo via modelos (workers).

O uso de Gunicorn permite concorrência, porém a configuração não está explicitamente definida. Para produção, parametrizar GUNICORN_WORKERS, usar processos/threads conforme carga e suportar workers para tarefas background se necessário.

**9. Descartabilidade:** inicialização rápida e desligamento limpo.

A inicialização é controlada, mas melhorias são necessárias para garantir desligamento seguro e rápido.

**10. Paridade Dev/Prod:** minimizar gap entre dev, staging e prod.

Embora haja uso de containers, ainda existem diferenças entre ambientes que devem ser reduzidas. Docker e compose fornecem boa paridade local. Mas docker-compose monta código via volume (dev-only) e tem segredos em arquivo. Para melhorar seria bom usar imagens idênticas entre ambientes, variáveis externas e infra como código para reduzir diferenças.

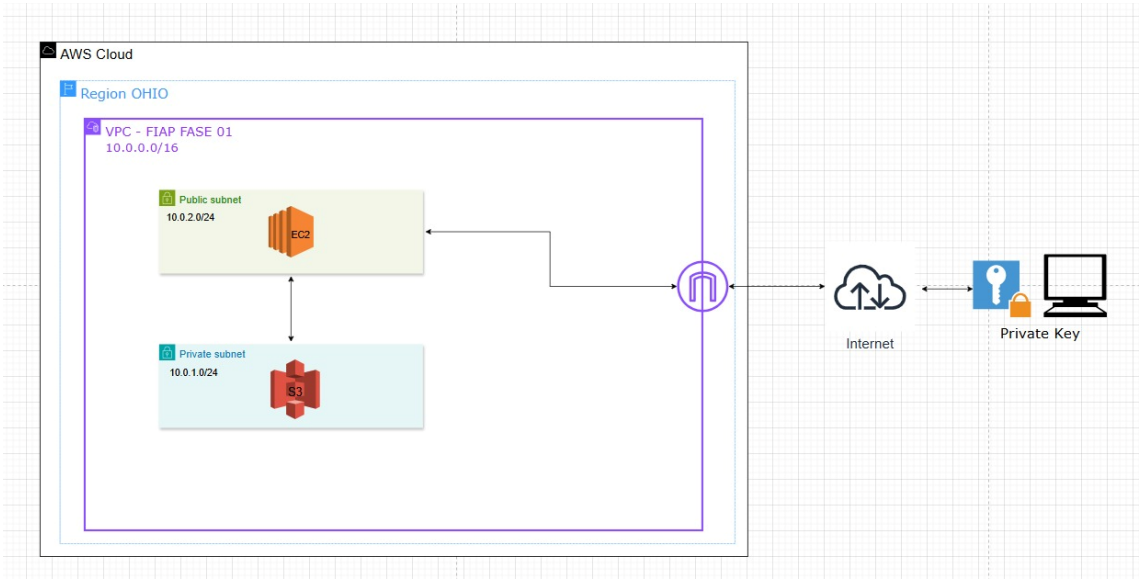
**11. Logs:** tratar logs como stream de eventos para captura por serviços.

Os logs são enviados para a saída padrão, mas recomenda-se integração com sistemas de observabilidade. A aplicação escreve em stdout/prints e Flask/gunicorn logam para stdout, mas para produção seria melhor integrar com um coletor (CloudWatch/ELK) e evitar escrever logs em arquivos locais.

**12. Processos de administração:** executar tarefas administrativas ad-hoc com ambiente do release.

Existem comandos administrativos, porém a execução automática deve ser evitada em produção.

**Diagrama de Arquitetura**



# Estimativa de custos da AWS



Entre em contato com seu representante da AWS: Entre em contato com a equipe de vendas

Data de exportação: **04/05/2026**

Idioma: **Português**

Estimar URL: **https://calculator.aws/#/estimate?id=abdbe1b77a6bd3a1468203ded12dc163f6bfa355**

## Resumo da estimativa

| Custo inicial | Custo mensal | Custo total de 12 months |
|---|---|---|
| 0,00 USD | 36,76 USD | 441,12 USD<br>Inclui um custo inicial |

### Estimativa detalhada

| Nome | Grupo | Região | Custo inicial | Custo mensal |
|---|---|---|---|---|
| **Amazon EC2** | Nenhum grupo aplicado | US East (Ohio) | 0,00 USD | 3,80 USD |
| **Status:** -<br>**Descrição:**<br>**Resumo da configuração:** Locação (Instâncias compartilhadas), Sistema operacional (Linux), Carga de trabalho (Consistent, Número de instâncias: 1), Instância do EC2 avançada (t3.micro), Pricing strategy (Compute Savings Plans 3yr No Upfront), Habilitar monitoramento (desabilitada), DT Entrada: Not selected (0 TB por mês), DT Saída: Not selected (0 TB por mês), DT Intrarregião: (0 TB por mês) | | | | |
| **Amazon RDS for MySQL** | Nenhum grupo aplicado | US East (Ohio) | 0,00 USD | 32,96 USD |
| **Status:** -<br>**Descrição:**<br>**Resumo da configuração:** Quantidade de armazenamento (20 GB), Armazenamento para cada instância do RDS (SSD de uso geral (gp3)), Nós (1), Modelo de preço (OnDemand), Tipo de instância (db.t3.micro), Utilização (somente sob demanda) (24 Hours/Day), Opção de implantação (Single-AZ) | | | | |

**Confirmação**

A Calculadora de Preços da AWS fornece apenas uma estimativa de suas taxas da AWS e não inclui nenhuma taxa aplicável. Suas taxas reais dependem de vários fatores, inclusive de seu uso real dos serviços da AWS. Saiba mais